

String

"Hello", 'Hi', 'Alice'

↳ String is an immutable sequence of characters.

"Hello"
 ('H', 'e', 'l', 'l', 'o')
 0 1 2 3 4

Indexing: index to get a character in a string

my_string = "ABCDE"
 -5 -4 -3 -2 -1
 0 1 2 3 4

print(my_string[0]) → A

print(my_string[-1]) → E

your_string = "Hello World!"
 0 1 2 3 4 5 6 7 8 9 10 11

print(your_string[6]) → W

Slicing

↳ Two steps process

1) Create a range of indexes

2) pull all those characters/values from those indexes

* It works on string, list, and tuple.

* It works on string, list, and tuple.

Syntax: `str[start:end:step]`

Ex: `"Hello World!"[1:7:2]`

0 1 2 3 4 5 6 7 8 9 10 11

↑ end before

1) `[1, 3, 5]`

2) `el` ← space

`my_str = "ABCD EXYZ"`

`print(my_str[0:4:1])` → `ABCD`

1) `[0, 1, 2, 3]`

2) `ABCD`

a new string.

`my_list = [1, 2, 3, 4, 10, 11, 12, 13, 14]`

`print(my_list[2:5:2])` → `[3, 10]`

1) `[2, 4]`

2) `[3, 10]`

a new list!

Default values with slicing

- 1) No step: `x[start:end:]`, `x[start:end]`
↳ default step = 1
↑ empty
- 2) No start: `x[:end:step]`
↳ default start = the first character (0, -1)
if step is neg.
- 3) No end: `x[start::step]`
↳ default end = the last character.

↳ default end = the last character (including)

Ex

str = "ABCDEFGH"
0 1 2 3 4 5 6
print(str[:3]) → ABC
↑ start/step → default

1) 0, 1, 2

2) ABC

print(str[4::2]) → EG

1) 4, 6

2) EG

print(str[::-1]) → GFE DCBA

↑ start = -1 backward

1) -1, -2, -3, -4, -5, -6, -7

2) GFE DCBA

Ex

names = "Patty_Lynn_Smith"
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

First = names[:5] → Patty

second = names[6:10] → Lynn

third = names[11:]
smith

For loop with string

len(str) → # character

my_str = "Hello"

for i in range(len(my_str)):

```
for i in range(len(my_str)):
```

```
    print(i, my_str[i])
```

① ② ③ ④ ⑤

loop each
character
in a string

```
for char in my_str:
```

```
    print(char)
```

→ 0 H
1 e
2 l
3 l
4 o

H
e
l
l
o

EX Count a character in a string

target = 't'

my_str = "+++ TT tt"

count = 0

```
for c in my_str:
```

```
    if c == target:
```

```
        count += 1
```

```
print(count)
```

→ 15

String Operators

• + : concatenate two string

↳ "A" + "B" → "AB"

"1" + "20" → "120"

• * : repeat a string n times

↳ "AB" * 3 → "ABABAB"

"1" * 4 → "1111"

• in : check whether a string is in another string

↳ True / False

("AB" in "CACAB") → True

("AB" in "cacab") → False

("a" in "xyz") → False

String Methods → Syntax: string.method(name)

1) String testing methods

• islower(): True if all characters are lowercase

• isupper(): True if all characters are uppercase

• isspace(): True " " spaces
(and '\n')

• isalnum(): True " " alphabet
or number

• isalpha(): True " " alphabet

• isdigit(): True " " numbers

• is digit(): true $\longrightarrow$ numbers

Ex

"123".is digit() $\longrightarrow$ True

"12a".is digit() $\longrightarrow$ False

"12ab".is alnum() $\longrightarrow$ True

"12ab"[(0,1)].is digit()

"12".is digit() $\longrightarrow$ True

2) Creating new string methods

• lower() $\longrightarrow$ return a new string in all lowercase

• upper() $\longrightarrow$ return a new string in all uppercase

Ex1

"abc".lower() $\longrightarrow$ "abc"

Ex2

my_str = "HeLLo"

your_str = my_str.lowercase() $\longrightarrow$ "hello"

print(my_str, your_string)

HeLLo hello

Strip() family

$\hookrightarrow$ string methods that return string

↳ string methods that return string without spaces in front of or at the end of string.

- `rstrip()` → remove all spaces at the end of string
- `lstrip()` → remove all spaces at the front of string
- `strip()` → remove all space on both sides

EX

```
str = " Hello World "  
print("A", str.lstrip(), "X")
```

A Hello World X

```
print("A", str.rstrip(), "X")
```

A Hello World X

```
print("A", str.strip(), "X") → A Hello world X
```

strip methods can remove characters

- `strip(char)` → remove all char in front/back of the string

EX

```
"aaabbccdda".strip(a)
```

bbccdda

```
str = "aaabbccdda"
```

```
print str.lstrip(a).lstrip(b).strip(a)
```

...
 print(str.lstrip('a').rstrip('b').strip('a'))
 "bbccadda".rstrip('b')
 "ccadda".strip('a')
 "ccadd"
 "ccadd"

Finding and Replacing methods

• find(str) : return the first index of str in the string
 ↳ if str not in the string, it will return -1

↳ like ctrl+f

"Hello world!".find("Wo") → 6
 0 1 2 3 4 5 6

"Hello".find("a") → -1

• replace(old string, new string)

↳ replace all instance of old string in the main string with a new string.

"A A A A A" replace('a', 'x')

"AaAaAaAaAa".replace('a', 'x')

↓
"AxAxAx AxAx"

"A A A".replace(' ', '') ← empty strings

↓
"AAA"

↑ remove by replacing

Split(^{splitter}) method

↳ split a string based on a splitter str
into a list of string

EX

"A-B-C-D".split('-')

↓ ↓ ↓ ↓
['A', 'B', 'C', 'D']

"10 // 20 // 30 // 40".split('//')

↓ ↓ ↓ ↓
['10', '20', '30', '40']

EX

Turn user's inputs which is
a sequence of numbers separated by
commas

("1, 2, 10, 20")

... and then calculate

into a list of number and then calculate
input_string = input("Enter numbers!")

↓ "1; 2; 10; 20"
numbers = input_string(',')
↓ ['1', '2', '10', '20']

for i in range(len(numbers)):

numbers[i] = int(numbers[i])

↑ convert string into int!

print(numbers) → [1, 2, 10, 20]

print(sum(numbers)/len(numbers)) → 33/4

join(list) method

↳ concatenate all items in the list
using the string

"x".join(['A', 'B', 'C'])
↓
A x B x C

→ Convert a list of str into a string

By default, `split()` method splits at space

"Hello World How are you?". `split()`

↓
["Hello", "World", "How", "are", "you?"]